

BMSSPy: A Python Package and Empirical Comparison of Bounded Multi-Source Shortest Path Algorithm

Connor Makowski, Willem Guter, Tim Russell, Lucas Castro, Austin Saragih
Massachusetts Institute of Technology, Cambridge, MA, USA {conmak,wjguter,trussell,saragih}@mit.edu
Federal University of Amazonas, Manaus, AM, BRA lucas.castro@icomp.ufam.edu.br

We introduce BMSSPy, the first open-source Python package that faithfully implements the Bounded Multi-Source Shortest Path (BMSSP) algorithm. This algorithm was recently discovered as a breakthrough that addresses Dijkstra’s sorting barrier for single-source shortest paths. BMSSPy implements the theoretical breakthrough with three practical advantages. First, it preserves the algorithm’s asymptotic performance guarantees while returning both distance labels and paths. Second, it clarifies pivot selection, tight-edge forest construction, and frontier management. Third, it handles graphs with non-unique path lengths, structural irregularities, unreachable vertices, and highly variable node degrees, ensuring function and robust performance on real data. With these three advantages, empirical experiments on grid and geospatial networks demonstrate that BMSSPy delivers competitive performance. For instance, on a graph with 200,000 nodes and 1.56 million edges, BMSSP runs $1.6\text{--}1.9\times$ faster than NetworkX’s Dijkstra and up to $3.3\text{--}3.9\times$ faster than igraph’s, although the highly optimized SCGraph Modified Dijkstra still achieves the best performance. BMSSP proves practical and efficient in practice, despite the remaining gap between theory and system-level performance. BMSSPy serves to support future research and development in shortest-path algorithms.

Key words: Software, Algorithms, Shortest Path, Logistics, Distance Estimation

1. Introduction

Solving the single-source shortest path (SSSP) problem on (un)directed graphs with non-negative weights is a foundational challenge in graph theory, operations research, theoretical computer science, and management science (Elmaghraby 1970, Orlin et al. 2010, Duan et al. 2025). Given m edges and n nodes, the classic algorithm by Dijkstra (1959), enhanced by modern heap structures, achieves $O(m + n \log n)$ runtime. The term $O(n \log n)$ arises from the necessity of sorting distances—a limitation known as the *sorting barrier*.

This barrier is fundamental and makes Dijkstra’s algorithm uniquely optimal when putting vertices in the exact order of increasing distance (Haeupler et al. 2024). However, recent breakthroughs reveal that this barrier is not essential if only distance values are required. The *Bounded Multi-Source Shortest Path* (BMSSP) algorithm (Duan et al. 2025) overcomes this, attaining $O(m \log^{2/3} n)$ complexity by eliminating global sorting and instead processing nodes within bounded distance ranges, resulting in faster performance on sparse graphs. Given arbitrary graphs and the need to convert to constant degree graphs, this is only true for sparse graphs, like many road networks.

Although BMSSP marks a significant theoretical advance, no verified public implementation currently exists. This lack of an accessible software package has hindered reproducibility and practical evaluation. Moreover, several details are left implicit in the theoretical descriptions, e.g., constructing the tight-edge forest, handling ordering when path lengths are not unique, managing the suggested data structure, and coping with unreachable nodes or variable degree distributions. Therefore, a working implementation is essential to address these details, making the algorithm accessible and comparable to current practical methods, particularly in real-world transportation networks (Zhan and Noon 1998). To that end, we have the following three research questions:

- **RQ1:** How can BMSSP be implemented in a robust and reproducible manner?
- **RQ2:** How can we resolve ambiguities that arise in practical implementations?
- **RQ3:** How does BMSSP perform relative to both vanilla and optimized Dijkstra variants on stylized grid graphs and real geospatial networks?

We address these questions with *BMSSPy*, the first fully open-source Python implementation of BMSSP. Our package transforms the theoretical framework into a practical and extensible tool. The key contributions are:

- We provide a *reproducible* specification of BMSSP that preserves theoretical runtime guarantees, outputs both shortest-path distances and explicit routes, and *resolves* implicit steps such as tight-edge forest construction, pivot selection, and all needed data structure operations (insert, pull, batch-prepend).
- We benchmark *BMSSPy* against popular Dijkstra implementations in *NetworkX*, *igraph*, and the high-performance *SCGraph*. On grid and real-world networks, *BMSSPy* can be competitive and outperform *NetworkX* and *igraph*, while *SCGraph* remains the fastest overall, underlining both the promise of BMSSP and the gap between theoretical and practical speed.

The rest of the paper is organized as follows: Section 2 surveys related literature and existing tools; Section 3 introduces *BMSSPy*, its interface, and usage; Section 4 presents implementation details, theoretical alignment, design choices, practical relaxations, operability safeguards, and efficiency improvements; Section 5 reports numerical benchmarks, including break-even analyses with Dijkstra baselines; Section 6 concludes with a brief summary and open directions.

2. Related Literature and Software

We briefly review relevant literature in Section 2.1 and available software in Section 2.2.

2.1. Related Literature

The single-source shortest path (SSSP) problem on directed graphs with nonnegative weights is one of the most extensively studied problems in algorithms. The classical solution is Dijkstra’s algorithm (Dijkstra 1959), which runs in $O(m + n \log n)$ time when implemented with Fibonacci heaps (Fredman and Tarjan 1987), relaxed heaps (Driscoll et al. 1988), or modern priority queues (Chen et al. 2007). Faster runtimes are possible when edge weights have additional structure. For undirected graphs with positive integer weights, Thorup (1999) shows a linear-time or $O(m)$ algorithm. For directed graphs with positive integer weights, Thorup (2003) shows a running time of $O(m + n \log \log \min\{n, C\})$, where C is the maximum edge weight. If the graph contains only K distinct edge weights, Orlin et al. (2010) devise a modified Dijkstra algorithm that runs in $O(m)$ time when $nK \leq 2m$ and in $O(m \log \frac{nK}{m})$ otherwise. In all these classical approaches, the $\log n$ factor reflects the need to order candidate distances, a limitation often referred to as the *sorting barrier*.

Recent work clarifies when the sorting barrier is unavoidable. Haeupler et al. (2024) show that Dijkstra is *universally optimal* whenever shortest paths must be output in increasing-distance order, even under powerful heap models. This establishes that sorting is inherent to the ordered-output version of SSSP. Practical variants such as A* do not improve worst-case complexity, but can yield major speedups in structured settings like road networks with Euclidean heuristics (Hart et al. 1968).

From the early survey of Dreyfus (1969) through the development of BMSSP, Dijkstra’s algorithm has been viewed as the most effective general-purpose method for SSSP. When the output order of vertices is not required, however, its sorting barrier can be avoided. Duan et al. (2025) develop BMSSP, which attains deterministic $O(m \log^{2/3} n)$ time by restricting relaxations to bounded distance ranges and reducing global sorting. This algorithm combines ideas from Bellman–Ford (Bellman 1958, Ford 1956) and Dijkstra. Earlier empirical work by Golden (1976) shows cases where Bellman–Ford outperformed Dijkstra on sparse Euclidean graphs. In this context, BMSSP represents the first deterministic SSSP method that circumvents full sorting while supporting directed and undirected graphs with nonnegative real weights.

Our work contributes to the practice of SSSP by translating the BMSSP framework into a concrete Python implementation and evaluating its performance on benchmarks from operations research and transportation.

2.2. Related Software

- **NetworkX:** A popular Python library for general graph analysis, offering Dijkstra’s Dijkstra (1959) and A^* Hart et al. (1968) implementations. It prioritizes usability and flexibility, though its pure Python design limits its efficiency on large graphs (Hagberg et al. 2008).
- **igraph:** Provides high-performance, C-backed algorithms with Python and R bindings (Csardi and Nepusz 2006). *igraph* excels at classical graph primitives but does not target geospatial applications or implement BMSSP.
- **SCGraph:** A dependency-free Python package focused on road, rail, and maritime routing (Makowski et al. 2025). *SCGraph* delivers optimized Dijkstra and A^* methods for large sparse networks, with bundled geospatial datasets geared for practical routing.

To our knowledge, no prior public implementation contains all components needed for BMSSP to realize its theoretical asymptotic guarantees. Castro et al. (2025) provide a promising C++ prototype that relies on hash tables and reports expected rather than worst-case running times. Together with this team, we are able to jointly identify a potential hashmap substitution that runs in $O(1)$ time. Valko et al. (2025) also recently released a Rust implementation evaluated on sparse payment-channel networks such as the Lightning Network. Their version omits several data-structural elements required for the theoretical bounds, which reduces the algorithm to a Dijkstra equivalent worst case runtime. Their experimental results are consistent with this: they observe only small or even negative speedups relative to Dijkstra, and attribute this to recursion overhead, constant factors, and memory-access costs.

Together, these efforts highlight a clear gap between the BMSSP theory and a practical, correct, and fully optimized implementation. Existing Python graph libraries provide strong baselines but do not yet fully support BMSSP. Our *BMSSPy* framework addresses this gap by delivering a reliable and empirically validated implementation that enables reproducible comparisons with state-of-the-art Dijkstra solvers on grid and geospatial graphs that are central to logistics and transportation applications.

3. The BMSSPy Package

We first describe the package in Section 3.1, then show how to install it (Section 3.2), and how to use it (Section 3.3).

3.1. Brief Description of BMSSPy

BMSSPy (Makowski 2025) is the first practical Python implementation of the *Bounded Multi-Source Shortest Path* (BMSSP) algorithm (Duan et al. 2025). Designed to bridge theory and practice, it supports directed graphs with nonnegative edge weights, returning both shortest path *distances* and *predecessors* to enable path reconstruction. Key features include:

- Support for multiple sources: input can be a single origin or a set of origin nodes.
- Returns distances and predecessor arrays for explicit paths.
- Robust handling of unreachable nodes and graphs with variable node degrees.
- Transparent implementation of implicit theoretical steps, such as tight-edge forest construction, pivot selection, and data structure operations.

3.2. Installation

BMSSPy is available on PyPI and requires Python 3.11 or later:

```
pip install bmsspy
```

3.3. Usage Example

The following example creates a small directed graph and demonstrates two use cases: computing distances and predecessors from a single source, and extracting a shortest path to a destination. It has the following inputs:

- **graph**: list of dictionaries. For node u , `graph[u]` maps neighbor v to a nonnegative weight w . Undirected graphs can be modeled by inserting edges in both directions.
- **origin_id**: an integer node id or a set of ids. If a set is given, all listed nodes are treated as sources with zero distance.
- **destination_id**: an integer node id or `None`. If `None`, the solver just returns the distance matrix and predecessor matrix.

The solver returns a dictionary with `origin_id`, `destination_id`, and the following:

- **distance_matrix**: list of distances from the origin (or origin set). Unreachable nodes have distance `inf`.

- **predecessor**: list of predecessors for path reconstruction. Unreachable nodes have predecessor -1.
- **path**: list of node ids on the shortest path when a destination is provided, else `None`.
- **length**: length of the returned path when a destination is provided, else `None`.

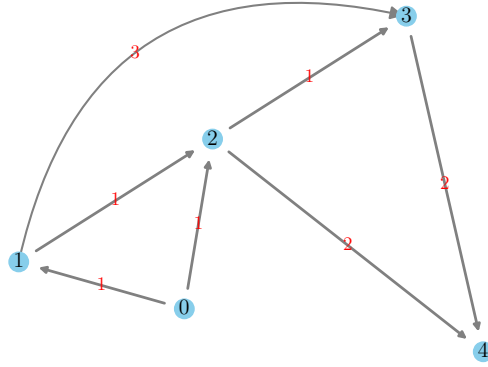
Example. The following code example builds a small directed graph (visualized in Figure 1), computes all shortest-path distances and predecessors using `bmssp`, and then extracts the shortest route to a specified destination. For multi-source queries, you can provide a set of origins; the solver initializes distance zero at each source and returns the shortest distances from the set. If you specify a destination, the resulting path starts at the origin with the minimum distance to that target. The next section details the implementation.

```
from bmsspy import Bmssp

# Graph with 5 nodes: 0..4
# Adjacency-list representation with nonnegative weights
graph = [
    {1: 1, 2: 1},    # 0 -> 1 (1), 0 -> 2 (1)
    {2: 1, 3: 3},    # 1 -> 2 (1), 1 -> 3 (3)
    {3: 1, 4: 2},    # 2 -> 3 (1), 2 -> 4 (2)
    {4: 2},          # 3 -> 4 (2)
    {}               # 4 has no outgoing edges
]
bmssp_graph = Bmssp(graph) # Initialize the graph as a Bmssp graph

# Distances and predecessors from origin 0
res_0 = bmssp_graph.solve(origin_id=0)
print(res_0) #=>
# {
#     'origin_id': 0,
#     'destination_id': None,
#     'predecessor': [-1, 0, 0, 2, 2],
#     'distance_matrix': [0, 1, 1, 2, 3],
#     'path': None,
#     'length': None
# }

# Shortest path from 0 to 4
res_0_4 = bmssp_graph.solve(origin_id=0, destination_id=4)
print(res_0_4) #=>
# {
#     'origin_id': 0,
#     'destination_id': 4,
#     'predecessor': [-1, 0, 0, 2, 2],
#     'distance_matrix': [0, 1, 1, 2, 3],
#     'path': [0, 2, 4],
#     'length': 3
# }
```

Figure 1 Visualization of the 5-node Example Graph

4. Functional Implementation of BMSSP

This section describes our practical and faithful implementation of the Bounded Multi-Source Shortest Path (BMSSP) algorithm from Duan et al. (2025), which is implemented in the *BMSSPy* package. We retain the original recursive structure and algorithm numbering while addressing missing details and adapting the method for practical use on real-world graphs. At a high level, our key contributions are:

1. **End-to-End Framework (E2E)**: We provide a complete workflow returning shortest path distances and predecessor arrays, supporting explicit path reconstruction for any given graph, origin set, and destination.
2. **Predecessor Tracking (PT)**: We explicitly show how to maintain a predecessor array, enabling the construction of full shortest paths beyond simple distance outputs.
3. **Relaxation of Uniqueness Assumption (RUA)**: The original algorithm assumes that all shortest path lengths are unique. The original paper offers a brief explanation that this does not lose generality after Lemma 2.1 in Duan et al. (2025). We implement this by using a distance value, a counter for nodes traversed, and a unique edge value system to provide lexicographic ordering.
4. **Explicit Forest and Pivot Construction (EFPC)**: We explicitly construct the tight-edge forest and derive the pivot set with rigorous criteria, clarifying previously implicit algorithmic steps.
5. **Fixes, Improvements, Notations (FIN)**: We implement several notation clarifications and bug fixes that address theoretical and practical issues, ensuring reliable performance on arbitrary graphs.

We first address preliminary assumptions in Section 4.1. Throughout the algorithmic descriptions in Section 4.2 (main algorithms) and Section 4.3 (end-to-end framework), we explicitly indicate where each enhancement is applied. The underlying data structure is discussed in Section 4.4. All notation follows Table 4.

4.1. Handling Assumptions

The original paper assumes two properties of the input graph: (1) all path distances are unique, and (2) the graph has constant degree. The original paper also inherently assumes that common "set" operations happen in $O(1)$ time. In this section, we describe how we address these assumptions so that the algorithm can operate on arbitrary, positively weighted graphs.

4.1.1. Unique Distances The algorithm requires all path distances to be unique. In the original formulation, this uniqueness is ensured by augmenting each distance value with additional tie-breaking information, such as a counter or endpoint identifiers. Conceptually, one may represent each distance as a tuple

$$dist[v] = (length, path_size, edge_id),$$

where *length* is the cumulative path weight, *path_size* records the number of edges used, and *edge_id* uniquely identifies the most recently added edge. During relaxation, this tuple updates via

$$dist[v] \leftarrow (dist[u][0] + G[u][v], dist[u][1] + 1, G_{edge}[u][v]).$$

This representation enables lexicographic comparison of distance values and guarantees uniqueness. The original paper proposes a slightly larger composite key:

$$dist[v] = (length, path_size, u, v),$$

where the final two components serve as a canonical tie-breaking mechanism.

While tuple-based lexicographic comparison is conceptually clean, it is operationally expensive: creating and accessing each tuple adds operational overhead, every comparison becomes a multi-field comparison, and auxiliary algorithms such as median-of-medians (used internally in BMSSPy) incur additional overhead and require algorithmic modifications. To avoid this, we encode the tuple as a *single scalar value* while preserving the exact lexicographic order.

We first round $length$ to a fixed number of digits provided as $precision$. Then we shift the decimal representation of $path_size$ and $edge_id$ sufficiently far to the right so that the three components never overlap. Let

$$\begin{aligned} counter &= 10^{-(precision + \lceil \log_{10}(2n+1) \rceil)} \\ edge_adj_value &= 10^{-(precision + \lceil \log_{10}(2n+1) \rceil + \lceil \log_{10}(m+1) \rceil)} \end{aligned}$$

Note that we use $2n + 1$ to allow simple rounding of the distance to the precision level. If instead we chose to truncate the distance to that precision level, we could use $n + 1$. Continuing, the scalar distance is then

$$\begin{aligned} dist[v] &= \text{round}(length, precision) \\ &\quad + path_size \cdot counter \\ &\quad + edge_id \cdot edge_adj_value. \end{aligned}$$

Because each edge has a fixed unique identifier, we preprocess $G_{edge}[\cdot][\cdot]$ so that it already stores the values $\{edge_adj_value, 2 \cdot edge_adj_value, \dots, m \cdot edge_adj_value\}$. During relaxation, we update

$$\begin{aligned} cdist[v] &\leftarrow cdist[u] + G[u][v] + counter \\ dist[v] &\leftarrow cdist[u] + G[u][v] + counter + G_{edge}[u][v], \end{aligned}$$

where $cdist$ omits the edge id portion, allowing us to skip rounding operations at each relaxation.

Because this approach relies on large decimal shifts, it is susceptible to floating-point error. Therefore, all computations use Python's `Decimal` type to guarantee exact precision. Future implementations may revisit specialized tuple structures to remove the need for fixed precision rounding.

4.1.2. Constant-Degree Transformation The algorithm's stated worst-case running time assumes that the input graph is constant degree. If the original graph does not satisfy this, we first transform it into an equivalent constant-degree graph. The conversion procedure is given in Appendix 8. After computing the shortest path in the transformed graph, we translate the resulting path back into the original graph in Appendix 9. From this point onward, unless explicitly noted, we assume all input graphs used in this section are constant degree.

4.1.3. Set Functions Sets are used extensively in the algorithms below. A naive implementation using hashmaps would yield only expected rather than worst-case running times. To ensure worst-case deterministic performance, we instead implement each set as a simple list augmented with a global counter and a global membership-tracking array. These components jointly provide $O(n)$ one-time initialization, $O(1)$ membership tests, and $O(|S|)$ set-extension operations. This approach is feasible because the universe of possible set elements has fixed size $|G|$, and every input element x is an integer that satisfies $0 \leq x < |G|$.

For a set S_a , let $scnt_a \leftarrow -1$ denote the global set counter and let $memb_a[0..|G|-1] \leftarrow -1$ be the global membership-tracking array, indexed by input elements x . For each location in the algorithm that we use a set S_a , we need to create the $scnt_a$ and $memb_a$ variables once. Then, each time we would have initialized an empty set, we instead apply $S_a \leftarrow []$ and we increment $scnt_a$. The set itself will be stored as a list, for example, $S_a = [x_1, x_2, \dots]$.

For every element x , the entry $memb_a[x]$ stores the most recent $scnt_a$ for which x was inserted. When inserting an element x into a set, we check whether $memb_a[x] = scnt_a$. If the equality holds, then x is already present in S_a . If it does not hold, we append x to S_a and set $memb_a[x] \leftarrow scnt_a$. This constant-time equality test provides an $O(1)$ membership check without auxiliary cleanup and guarantees worst-case performance.

Set extension is performed analogously. To extend a set S_a with a set S_b , we iterate over the list representation of S_b , for example $S_b = [y_1, \dots, y_k]$, and for each element y_i we test whether $memb_a[y_i] = scnt_a$. If not, we append y_i to S_a and update $memb_a[y_i] \leftarrow scnt_a$. We note that set removal is not used in this paper and is therefore not discussed in detail here.

Expanding the structure by adding $V_a[0..|G|-1] \leftarrow 0$, we can create an equivalent $O(1)$ assignment and lookup dictionary. To assign a value y to a key x , just set $V_a[x] \leftarrow y$. Then update S_a and $memb_a$ using the aforementioned process. Retrieving the assigned value for a key x would require checking first for relevance $memb_a[x] = scnt_a$, and if true, returning $V_a[x]$. To invalidate a key x in the dictionary, it would be sufficient to set $memb_a[x] \leftarrow -1$. This invalidation, however, would not remove the x from the underlying S_a list. If you do not need to iterate over the dictionary, S_a can be omitted and removal becomes irrelevant.

It is important to note that each recursion level has its own set U in Algorithm 3 that is unique from other U sets at each recursion level. This requires creating one $memb$ list (with length n) and $scnt$ variable per recursion level l . Since l can be approximated as $\log(n)^{1/3}$ given the parameters in Algorithm 4, this yields a one-time initialization cost

of $O(n \log(n)^{1/3})$. This framework enables deterministic worst-case membership checks, set extension, and various dictionary operations while avoiding the variability inherent in hashmap implementations.

4.2. Main Algorithms

BMSSPy is organized around three main algorithms that closely follow Algorithms 1-3 in Duan et al. (2025).

Algorithm 1: Find Pivots. This subroutine expands the current frontier S under a distance bound B and selects a reduced subset of *pivots* $P \subseteq S$ for the next recursion level. It performs k rounds of bounded relaxations (Bellman–Ford style) to build a temporary frontier W of vertices with distance $< B$ from S . If W grows too quickly ($|W| > k|S|$), the routine short-circuits and sets $P = S$, capping the work passed downward. Otherwise, it constructs a *tight-edge forest* on W (via the predecessor relation) and chooses as pivots those $u \in S$ that are forest roots with subtree size at least k . These pivots summarize the influence of S in the current window, reducing redundant relaxations and avoiding global ordering while preserving the theoretical recursion and batch bounds.

Our Find Pivots implementation in BMSSPy refines several steps of the original theoretical description to ensure correctness and clarity on arbitrary weighted graphs:

- *Use of evolving frontier variables* (Lines 1–16): We replace indexed frontier arrays (W_i) with W_{curr} and W_{prev} for clarity and memory efficiency.
- *Counter-based unique distances* (Lines 6 and 10): We apply our lightweight tie-breaking mechanism described in Section 4.1.1 using *counter* and G_{edge} adjustments so that all effective distances remain unique. The relevant parameters are initialized in Algorithm 5.
- *Predecessor tracking* (Line 9): updates a global `pred[]` array defined in Algorithm 4 for path reconstruction and for later pivot determination in the tight-edge forest.
- *Explicit forest construction* (Lines 17–23): builds the adjacency-list representation of the tight-edge forest F directly from recorded predecessors rather than distance equalities.
- *Pivot root selection* (Lines 25–28): explicitly checks in-degree $u \notin F^{ID}$ to identify forest roots and calls ISPIVOT (Appendix - Algorithm 6) to determine whether their subtrees exceed the size threshold k .

Algorithm 1: Find Pivots (BmsspCore.find_pivots)**Input:** upper bound B , frontier S **Output:** pivot set P , temporary frontier W

```

1  $W \leftarrow S; W_{\text{prev}} \leftarrow S;$  // FIN: initialize previous frontier
2 for 1 to  $k$  iterations do
3    $W_{\text{curr}} \leftarrow \emptyset;$ 
4   foreach  $u \in W_{\text{prev}}$  do
5     foreach  $(v, w) \in G[u]$  do
6        $d' \leftarrow \text{cdist}[u] + w + \text{counter} + G_{\text{edge}}[u][v];$  // RUA: Unique distance
7       if  $d' \leq \text{dist}[v]$  then
8          $\text{dist}[v] \leftarrow d';$ 
9          $\text{pred}[v] \leftarrow u;$  // PT: update predecessor
10         $\text{cdist}[v] \leftarrow \text{cdist}[u] + w + \text{counter};$  // RUA: Counter distance update
11        if  $d' < B$  then
12           $W_{\text{curr}} \leftarrow W_{\text{curr}} \cup \{v\}$ 
13       $W \leftarrow W \cup W_{\text{curr}};$ 
14      if  $|W| > k|S|$  then
15        return  $(S, W)$ 
16       $W_{\text{prev}} \leftarrow W_{\text{curr}};$  // FIN: advance to next iteration frontier
17  $F \leftarrow \{\};$  // EFPC: initialize empty adjacency lists for forest  $F$ 
18  $F^{ID} \leftarrow \emptyset;$  // EFPC: initialize in-degree set  $F^{ID}$ 
19 foreach  $u \in W$  do
20   foreach  $(v, w) \in G[u]$  do
21     if  $v \in W$  and  $\text{pred}[v] = u$  then // EFPC: add edge  $v$  if reached from  $u$ 
22        $F[u] \leftarrow F[u] + [v]$  if  $u \in F$  else  $[v];$  // EFPC: record child  $v$  in parent  $u$ 
23        $F^{ID} \leftarrow F^{ID} \cup \{v\};$  // EFPC: record  $v$  has an incoming edge
24  $P \leftarrow \emptyset;$ 
25 foreach  $u \in S$  do
26   if  $u \notin F^{ID}$  then // EFPC: root node in forest  $F$ 
27     if  $\text{ISPIVOT}(u, F, k)$  then // EFPC: call Algorithm 6
28        $P \leftarrow P \cup \{u\};$  // EFPC: add pivot  $u$  to set  $P$ 
29 return  $(P, W);$ 

```

Algorithm 2: Base Case. At recursion depth $l = 0$, BMSSP reduces to a bounded single-source search from the unique $x \in S$. We run a Dijkstra-style exploration under the current bound B , discovering vertices with distance $< B$ whose shortest paths visit x . The search stops as soon as the $(k + 1)$ st vertex would be explored; its weight becomes the refined boundary B' , and the already-settled k vertices form the new frontier below B' . If fewer than $k + 1$ vertices are found, we return $(B', S') = (B, S')$. Our Base Case implementation in BMSSPy makes the following clarifications to ensure correctness:

- *Early termination at $k+1$* (Line 5): set B' to the weight of the would-be $(k+1)$ st vertex and stop, yielding exactly k completed vertices below B' . This is done slightly

differently than the original algorithm 2 in that we can avoid the extra operational overhead of filtering for only values lower than B' after the main while loop ends.

- *Counter-based unique distances* (Lines 8 and 12): similar to Algorithm 1.
- *Predecessor tracking* (Line 11): update $\text{pred}[]$ on improvement, as in Algorithm 1.

Algorithm 2: Base Case (`BmsspCore.base_case`)

Input: upper bound B , singleton frontier $S = \{x\}$

Output: new bound B' , new frontier S'

```

1  $Heap \leftarrow \text{BINARYHEAP}((\text{dist}[x], x)); S' \leftarrow \emptyset; B' \leftarrow B;$ 
2 while  $Heap \neq \emptyset$  do
3    $(d, u) \leftarrow Heap.\text{EXTRACTMIN}();$ 
4   if  $|S'| \geq k$  then
5      $B' \leftarrow d; \text{break};$                                      // FIN: stop at (k+1)st vertex
6    $S' \leftarrow S' \cup \{u\};$ 
7   foreach  $(v, w) \in G[u]$  do
8      $d' \leftarrow \text{cdist}[u] + w + \text{counter} + G_{\text{edge}}[u][v];$       // RUA: unique distance
9     if  $d' \leq \text{dist}[v]$  and  $d' < B$  then
10       $\text{dist}[v] \leftarrow d';$ 
11       $\text{pred}[v] \leftarrow u;$                                      // PT: update predecessor
12       $\text{cdist}[v] \leftarrow \text{cdist}[u] + w + \text{counter};$            // RUA: advance counter distance
13       $Heap.\text{INSERTORDECREASEKEY}(v, d');$ 
14 return  $(B', S');$ 

```

Algorithm 3: Recursive BMSSP. For $l > 0$, this routine coordinates the whole BMSSP step. The algorithm first compresses the frontier using `FINDPIVOTS` to obtain the pivot set P and a temporary frontier W . It then initializes a capacity-aware data structure with $M = 2^{(l-1)t}$. Following this, it inserts P into the data structure D . Then, while the work budget is not exceeded and the frontier is nonempty, it: (i) pulls a smallest-key batch $(B_{\text{curr}}, S_{\text{curr}})$, (ii) recurses one level down on this batch, (iii) relaxes edges out of the newly completed set U_{curr} , and (iv) reinserts improved neighbors into the correct distance window, either the current window $[B_{\text{curr}}, B]$ or the lower window $[B_{\text{comp}}, B_{\text{curr}})$ via `BatchPrepend`. It returns the completion bound B_{comp} and the union of all vertices completed below that bound. Given that we are swapping hashmap sets for our sets defined in Section 4.1.3, we cannot store tuples in our sets. Instead, we simply store relevant vertices in set K . We then `BatchPrepend` their vertices and distances as a list of tuples to match the original function. Our Recursive BMSSP in BMSSPy also incorporates *counter-based unique distances* (Lines 8 and 12), consistent with Algorithm 1, and *predecessor tracking* (Line 11), which updates $\text{pred}[]$ if needed, as in Algorithms 1 and 2.

Algorithm 3: Recursive BMSSP (`BmsspCore.recursive_bmssp`)

Input: depth l , bound B , frontier S
Output: (B_{comp}, U)

```

1 if  $l = 0$  then
2   | return  $(B_{\text{comp}}, U) \leftarrow \text{BASECASE}(B, S)$ ;
3  $(P, W) \leftarrow \text{FINDPIVOTS}(B, S)$ ;
4  $M \leftarrow 2^{(l-1)t}$ ;
5  $\text{BMSSPDATAStructure}(M, B)$ 
6 foreach  $p \in P$  do
7   |  $D.\text{INSERT}(p, \text{dist}[p])$ 
8  $B_{\text{comp}} \leftarrow \min(\{\text{dist}[p] : p \in P\} \cup \{B\})$ ;  $U \leftarrow \emptyset$ ;
9 while  $|U| < k \cdot 2^{lt}$  and  $\neg D.\text{ISEMPTY}()$  do
10  |  $(B_{\text{curr}}, S_{\text{curr}}) \leftarrow D.\text{PULL}()$ ;
11  |  $(B_{\text{comp}}, U_{\text{curr}}) \leftarrow \text{RECURSIVEBMSSP}(l-1, B_{\text{curr}}, S_{\text{curr}})$ ;
12  |  $U \leftarrow U \cup U_{\text{curr}}$ ;
13  |  $K \leftarrow \emptyset$ ;
14  | foreach  $u \in U_{\text{curr}}$  do
15    | foreach  $(v, w) \in G[u]$  do
16      |  $d' \leftarrow \text{cdist}[u] + w + \text{counter} + G_{\text{edge}}[u][v]$ ;           // RUA: Unique effective key
17      | if  $d' \leq \text{dist}[v]$  then
18        |  $\text{dist}[v] \leftarrow d'$ ;
19        |  $\text{pred}[v] \leftarrow u$ ;                                           // PT: Update predecessor
20        |  $\text{cdist}[v] \leftarrow \text{cdist}[u] + w + \text{counter}$ ;           // RUA: Counter distance update
21        | if  $B_{\text{curr}} \leq d' < B$  then
22          |  $D.\text{INSERT}(v, d')$ 
23        | else if  $B_{\text{comp}} \leq d' < B_{\text{curr}}$  then
24          |  $K \leftarrow K \cup \{v\}$ ;                                     // FIN: Store K as set of vertices
25  $D.\text{BATCHPREPEND}(\{(i, \text{dist}[i]) : i \in K \cup \{j : j \in S_{\text{curr}}, B_{\text{comp}} \leq \text{dist}[j] < B_{\text{curr}}\}\})$ ;
26  $B_{\text{comp}} \leftarrow \min(B_{\text{comp}}, B)$ ;
27 return  $(B_{\text{comp}}, U \cup \{x \in W : \text{dist}[x] < B_{\text{comp}}\})$ ;

```

4.3. End-to-End Framework

In the previous section, the BMSSP framework comprises three main algorithms: Find Pivots (Algorithm 1), Base Case (Algorithm 2), and Recursive BMSSP (Algorithm 3), where the latter coordinates the first two.

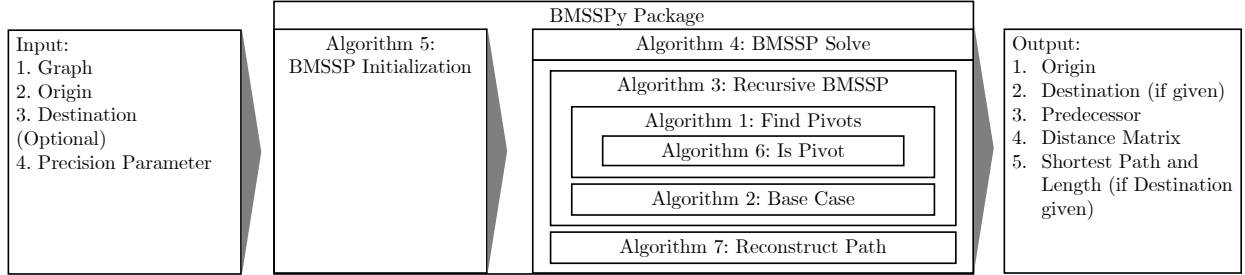
Our main contribution is an end-to-end implementation that utilizes BMSSP to compute shortest-path distances and reconstruct predecessor paths for a given graph G . As illustrated in Figure 2, the workflow proceeds as follows:

1. **Initialization** - Algorithm 5 preprocesses the input graph G (in adjacency-list form) and sets up the precision parameter precision , along with adjustment values (counter , G_{edge}) to ensure unique path distances.
2. **Solve phase** - Algorithm 4 initializes distance and predecessor arrays, sets parameters k , t , and l , and invokes the Recursive BMSSP (Algorithm 3).

3. **Post-processing** - Distances are rounded according to the specified precision level, and final paths are reconstructed using Algorithm 7 in the appendix (or Algorithm 9 in the appendix if the graph is converted to constant degree).

This end-to-end process returns both shortest-path distances and predecessor paths for the given origin set O and, optionally, a destination node $dest$. The following sections describe the remaining algorithms that complete the framework.

Figure 2 End-to-End BMSSPy Framework



Algorithm 4: BMSSP Solve. This is the main entry point of the framework. It initializes the distance, counter-distance, and predecessor arrays, sets algorithm parameters (k , t , and l using $\log_2$), and initialize structures for sets and dictionaries here (see Section 4.1.3 and 4.4). We then invoke the Recursive BMSSP (Algorithm 3). After computing distances, it rounds them back to the specified precision and optionally reconstructs the shortest path to a target node using Algorithm 7 in the appendix. Our implementation clarifies the $\log_2$ base used in parameter definitions and integrates explicit path reconstruction, returning both distance and path results.

Algorithm 5: BMSSP Initialization. This setup routine preprocesses the input graph and prepares adjustment constants needed to meet the unique path length requirements for the algorithm. It rounds edge weights to the desired precision p , computes unique offset parameters (`counter` and G_{edge}) to ensure unique distances for paths, and stores these within the BMSSP object. The resulting preprocessed graph supports deterministic, precision-consistent shortest-path calculations.

4.4. Data Structure

The data structure contained several ambiguities and open questions that required clarification during implementation. To support the required operations without changing their asymptotic runtime, we introduced several major and minor adjustments that enable the implementation, although some may be unnecessary under alternative designs.

Algorithm 4: BMSSP Solve (E2E) (BmsspCore.__init__ & Bmssp.solve)**Input:** origin set O , destination id $dest$ (optional)**Output:** dictionary with path, length, distance matrix, and predecessor

```

1  $dist[0..|G|-1] \leftarrow \infty$ ;
2  $cdist[0..|G|-1] \leftarrow \infty$ ; // RUA: Create counter distance matrix
3  $pred[0..|G|-1] \leftarrow -1$ ; // PT: Create predecessors track for path reconstruction
4 foreach  $o \in O$  do
5    $dist[o] \leftarrow 0$ ;
6    $cdist[o] \leftarrow 0$ ;
7  $k \leftarrow \lfloor \log_2(|G|)^{1/3} \rfloor$ ; // FIN: Use  $\log_2$ 
8  $t \leftarrow \lfloor \log_2(|G|)^{2/3} \rfloor$ ; // FIN: Use  $\log_2$ 
9  $l \leftarrow \left\lceil \frac{\log_2(|G|)}{t} \right\rceil$ ; // FIN: Use  $\log_2$ 
10  $(B, U) \leftarrow \text{RECURSIVEBMSSP}(l, \infty, O)$ ; // Algorithm 3
11 foreach  $i \in dist$  do
12   if  $dist[i] \neq \infty$  then
13      $dist[i] \leftarrow \text{round}(dist[i], precision)$ ; // RUA: Convert back to original
14      $precision$ 
14 if  $dest \neq \text{None}$  then
15    $path \leftarrow \text{RECONSTRUCTPATH}(dest, pred)$ ; // PT: Algorithm 7
16    $length \leftarrow dist[dest]$ ;
17 else
18    $path \leftarrow \text{None}$ ;
19    $length \leftarrow \text{None}$ ;
20 return dictionary with keys:  $O, dest, pred, dist, path, length$ ;

```

Algorithm 5: BMSSP Initialization (E2E) (Bmssp.__init__)**Input:** graph G (adjacency list), precision $precision$ **Output:** BMSSP object with preprocessed graph and edge adjustment values

```

1 if  $|G| < 2$  then
2   raise error "graph must have at least 2 nodes"
3 foreach  $node\ u \in G$  do
4   foreach  $(v, w) \in G[u]$  do
5      $G[u][v] \leftarrow \text{round}(w, precision)$ ;
6  $m \leftarrow \sum_{u \in G} |G[u]|$ ;
7  $n \leftarrow |G|$ ;
8  $precision \leftarrow precision$ ;
9  $counter \leftarrow 10^{-(precision + \lceil \log_{10}(2n+1) \rceil)}$ ;
10  $edge\_adj\_value \leftarrow 10^{-(precision + \lceil \log_{10}(2n+1) \rceil + \lceil \log_{10}(m+1) \rceil)}$ ;
11  $G_{edge}[0..|G|-1] \leftarrow \{\}$ ; // Can use hashmaps here since graph is constant degree
12  $edge\_id \leftarrow 0$ ;
13 foreach  $u \in G$  do
14   foreach  $(v, w) \in G[u]$  do
15      $edge\_id \leftarrow edge\_id + edge\_adj\_value$ ;
16      $G_{edge}[u][v] \leftarrow edge\_id$ ;

```

The first major addition is closely related to the mechanisms described in Section 4.1.3. We create l recursion level dictionaries that can perform lookup, assignment, and key invalidation. We need l dictionaries since each recursion level has its own state that is independent of the other recursion level states above it. These are used to access the linked list node as well as the appropriate data buffer in $O(1)$ time.

The second major addition is that all linked lists are doubly linked. This includes both the nodes within each block and the blocks within each chain.

We introduce several minor modifications or extensions to support full functionality:

- The linked-list blocks referenced by the red–black tree were generalized to chains of linked-list blocks. Although not required when using unique path lengths, this generalization is necessary for supporting non-unique values in future variants. It prevents any block from exceeding size m when more than m keys share the same value. Only the “head” block referenced by the red–black tree may contain values less than the lower bound, while all subsequent blocks in the chain contain only values equal to the lower bound. This ensures that pull operations remain $O(m)$ and that insert operations do not need to consider later blocks.
- A minimum block size of 2 is enforced, though the return size for the “pull” operation may be 1.
- Each block’s linked list has an associated container object that tracks its upper bound and current size.
- The block containing the global upper bound is never removed from the red–black tree, even if it is empty.

5. Numerical Experiments

We now evaluate the practical performance of BMSSP using our Python implementation, *BMSSPy*. This section addresses two questions. First, how does BMSSP compare to a range of Dijkstra implementations on both synthetic grid graphs and real geospatial networks? Second, given its better asymptotic complexity, at what (if any) problem size would BMSSP overtake a highly optimized *SCGraph* Dijkstra implementation? The first question is studied in Section 5.1, while the second is examined in the break-even analysis of Section 5.2 using the crossover plot in Figure 6.

5.1. Comparison of Run Times on Grid Graphs and Geospatial Graphs

We compare BMSSP (with and without the constant-degree conversion, BMSSP-CD and BMSSP respectively) against four Dijkstra baselines: *Vanilla Dijkstra*, a simple no heap pure-Python implementation; *SCGraph MD*, an optimized Dijkstra implementation through SCGraph; *Nx Dijkstra*, the implementation in NetworkX; and *iG Dijkstra*, the implementation in igraph. For each baseline, we chose the default multi-source supporting Dijkstra algorithm provided by the package.

Setup: We evaluate performance on two families of synthetic grid graphs. Each graph has a wall running from the top center to 5 units from the bottom of the graph, blocking nodes and arcs that would otherwise pass through that area. The first family contains square grids of sizes 15×15 up to 1000×1000 . The second contains rectangular grids of sizes 15×50 up to 100×2000 . For every grid instance, we run three origin cases: Bottom Left (BL), Top Right (TR), and Center (C). All experiments were executed on Ubuntu 24.04.3 using Python 3.12.3, running single-threaded on a AMD Ryzen™ Threadripper™ 3970X 32-core 4.5 GHz machine with 128 GiB RAM.

Figure 3 illustrates the resulting shortest-path tree structures on a 50×50 square grid for the BL, TR, and C cases when calculated using BMSSP-CD. Note that these trees exhibit alternative shortest paths near some edges. This is related to how the graphs are converted to constant degree graphs paired with our proposed lexicographic length calculations on those converted constant degree graphs.

We also include five large real geospatial networks from SCGraph and SCGraph_Data (Makowski et al. 2025): Maritime Network (Marnet), North America Rail, Oak Ridge Maritime, US Freeway, and World Highways and Marnet. For each of these geographs we use three origins: Los Angeles (LA), New York (NY), and Seattle (SEA). An example of a shortest-path tree from LA on the North America Rail network is shown in Figure 4.

Results. Tables 1, 2, and 3 report average runtimes and standard deviations for all graph–origin combinations over 10 iterations. Figure 5 visualizes these tables in absolute terms and as relative performance against BMSSP. We observe that SCGraph’s Modified Dijkstra (MD) is the fastest method on almost all instances. BMSSP is slower than SCGraph MD but consistently faster than vanilla Dijkstra. SCGraph and igraph implementations of Dijkstra perform well on smaller grids, yet their runtimes grow quickly as the grids become

Figure 3 Example Construction of Shortest-path Tree for 50×50 Square Gridgraph for Bottom Left (BL), Top Right (TR), and Center (C) Cases

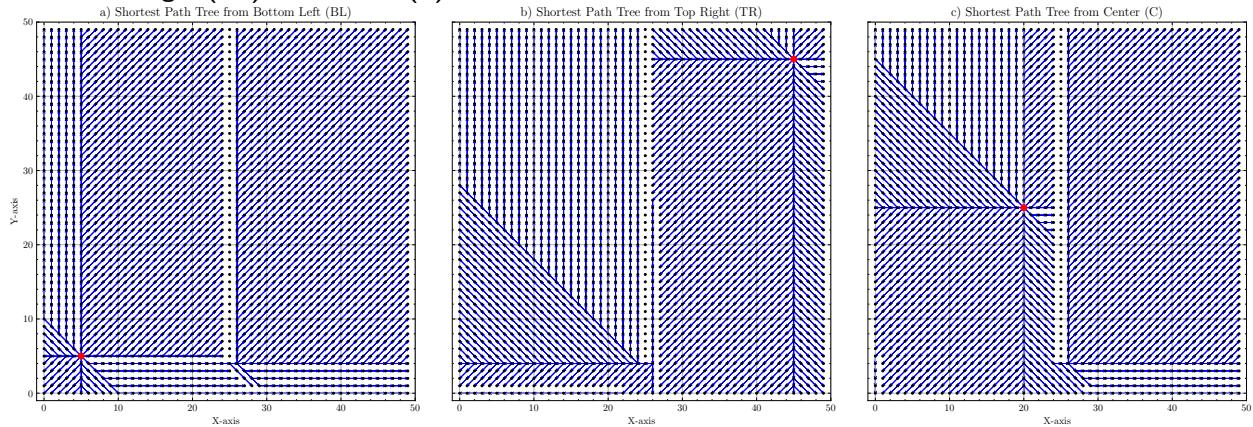
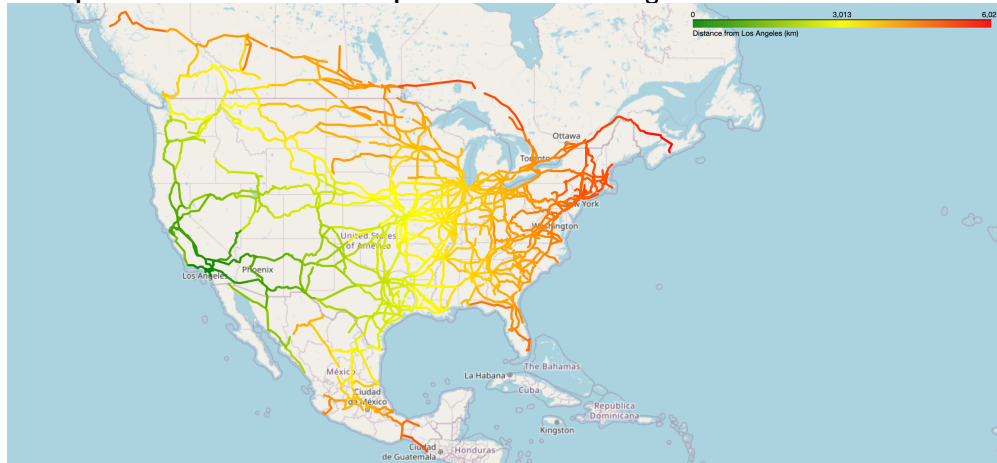


Figure 4 Example Construction of Shortest-path Tree from Los Angeles for North American Rail

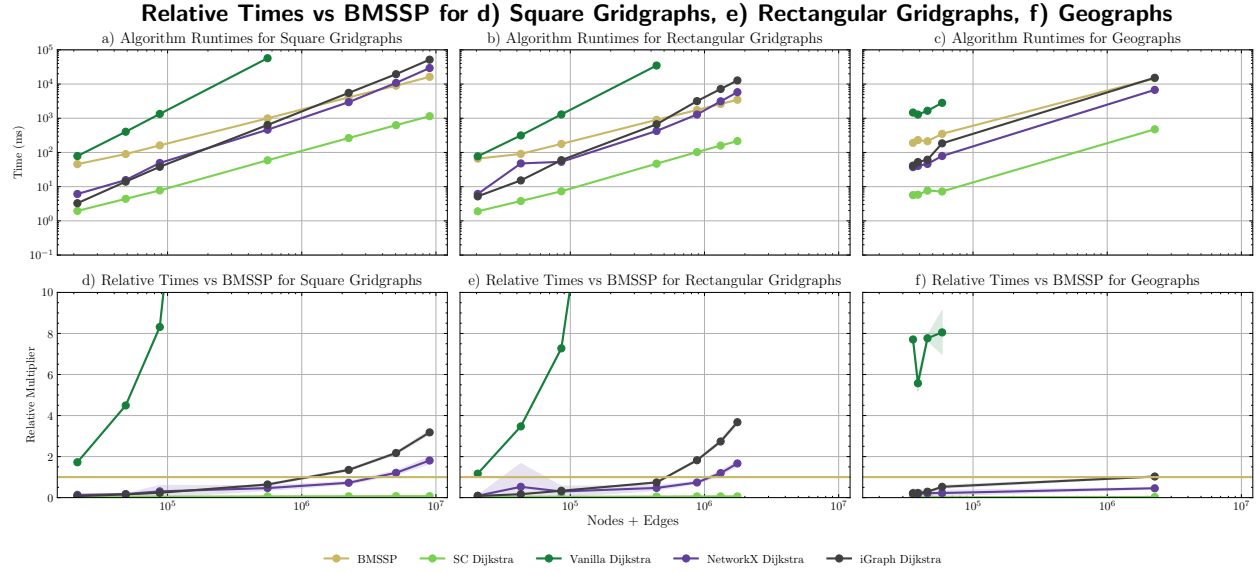


larger and more elongated. In the case of *NetworkX*, this longer runtime appears partly due to its default handling of multi-source problems, where it constructs paths for every possible node.

The comparison between *BMSSP* and *BMSSP-CD* requires care. To maintain consistency in the comparison, we report runtimes using the original graph sizes, despite the constant-degree transformation producing larger graphs. The constant-degree version is slower at small and medium sizes because the added nodes and edges create more overhead than the sorting savings can offset. As the graphs grow, one might expect sorting costs to dominate and eventually favor *BMSSP-CD*, but we do not see this given the graph sizes that we test. *BMSSP-CD* scales more steeply than *BMSSP*, and the extra node and edge expansion outweighs its theoretical advantages (again, for the grid sizes and connection patterns we test). It is also worth noting that the grid graphs themselves are already close

to constant degree, with most nodes having eight incoming and eight outgoing edges, so the transformation provides little structural benefit. Given this, we suspect that increasing the requirement for a constant degree 2 graph to some larger number based on the graph structure, paired with other work budget and algorithmic changes, could yield further improvements to the worst case and real world speeds.

Figure 5 Algorithm Runtimes for a) Square Gridgraphs, b) Rectangular Gridgraphs, c) Geographs



As the square and rectangular grids grow, BMSSP becomes faster on average than both *NetworkX* and *igraph*. In the large 200×1000 grids and 1000×1000 grids, BMSSP is roughly 1.6 to 2.1 times faster than *NetworkX* and about 2.4 to 4.2 times faster than *igraph*. Figure 5 illustrates this shift: the relative performance curves for *NetworkX* and *igraph* rise above BMSSP on the largest grids, while BMSSP continues to grow more slowly.

The geospatial graphs show a different pattern. These networks are sparse and irregular, so *SCGraph MD* again achieves the lowest runtimes. BMSSP, *NetworkX*, and *igraph* have similar runtimes on medium-sized geographs. On the largest geograph (World Highways and Marnet), *SCGraph MD* is the fastest, followed by *NetworkX*, *igraph* and then BMSSP.

Taken together, Tables 1–3 and Figure 5 show that BMSSP improves substantially vs Vanilla Dijkstra, becomes competitive with general-purpose libraries on large gridgraphs, and outperforms *NetworkX* and *igraph* on larger synthetic grids. At the same time, *SCGraph MD* remains the fastest overall due to its highly optimized implementation.

Table 1 Runtime Comparison on Square Gridgraphs (Mean $\pm$ Standard Deviation ms)

Graph	Case	Nodes	Edges	BMSSP-CD	BMSSP	Vanilla Dijkstra	SCGraph MD	Nx Dijkstra	iG Dijkstra
Square Grid 15 $\times$ 15	BL	225	1,484	27.96 $\pm$ 0.19	7.92 $\pm$ 0.05	0.83 $\pm$ 0.01	0.17 $\pm$ 0.01	0.42 $\pm$ 0.02	0.11 $\pm$ 0.03
Square Grid 15 $\times$ 15	TR	225	1,484	26.51 $\pm$ 0.12	7.71 $\pm$ 0.06	0.82 $\pm$ 0.01	0.16 $\pm$ 0.01	0.42 $\pm$ 0.02	0.12 $\pm$ 0.02
Square Grid 15 $\times$ 15	C	225	1,484	26.93 $\pm$ 0.11	7.84 $\pm$ 0.12	0.83 $\pm$ 0.01	0.16 $\pm$ 0.01	0.42 $\pm$ 0.02	0.11 $\pm$ 0.02
Square Grid 25 $\times$ 25	BL	625	4,424	77.03 $\pm$ 0.25	11.39 $\pm$ 0.14	5.32 $\pm$ 0.01	0.52 $\pm$ 0.01	1.41 $\pm$ 0.10	0.41 $\pm$ 0.05
Square Grid 25 $\times$ 25	TR	625	4,424	69.39 $\pm$ 0.20	13.85 $\pm$ 0.20	5.37 $\pm$ 0.06	0.47 $\pm$ 0.01	1.39 $\pm$ 0.10	0.49 $\pm$ 0.07
Square Grid 25 $\times$ 25	C	625	4,424	74.51 $\pm$ 0.14	11.59 $\pm$ 0.16	5.38 $\pm$ 0.02	0.49 $\pm$ 0.01	1.42 $\pm$ 0.09	0.42 $\pm$ 0.05
Square Grid 50 $\times$ 50	BL	2,500	18,774	322.46 $\pm$ 1.22	38.23 $\pm$ 0.48	78.08 $\pm$ 0.08	1.97 $\pm$ 0.01	6.04 $\pm$ 0.36	3.07 $\pm$ 0.21
Square Grid 50 $\times$ 50	TR	2,500	18,774	312.41 $\pm$ 1.17	56.81 $\pm$ 0.50	78.49 $\pm$ 0.12	1.92 $\pm$ 0.01	6.12 $\pm$ 0.40	3.80 $\pm$ 0.27
Square Grid 50 $\times$ 50	C	2,500	18,774	322.96 $\pm$ 0.96	41.42 $\pm$ 0.63	78.63 $\pm$ 0.12	1.97 $\pm$ 0.01	6.11 $\pm$ 0.34	2.97 $\pm$ 0.22
Square Grid 75 $\times$ 75	BL	5,625	43,124	718.83 $\pm$ 3.10	82.26 $\pm$ 0.34	403.02 $\pm$ 0.62	4.36 $\pm$ 0.02	15.22 $\pm$ 0.61	10.99 $\pm$ 0.44
Square Grid 75 $\times$ 75	TR	5,625	43,124	771.11 $\pm$ 188.49	98.64 $\pm$ 0.45	402.88 $\pm$ 0.62	4.36 $\pm$ 0.03	16.11 $\pm$ 0.78	19.53 $\pm$ 0.47
Square Grid 75 $\times$ 75	C	5,625	43,124	749.08 $\pm$ 46.84	88.26 $\pm$ 0.53	403.09 $\pm$ 0.85	4.49 $\pm$ 0.02	15.38 $\pm$ 0.65	11.45 $\pm$ 0.64
Square Grid 100 $\times$ 100	BL	10,000	77,474	1,312.97 $\pm$ 50.00	152.90 $\pm$ 0.25	1,353.53 $\pm$ 7.55	7.65 $\pm$ 0.06	48.08 $\pm$ 46.37	35.58 $\pm$ 0.36
Square Grid 100 $\times$ 100	TR	10,000	77,474	1,300.93 $\pm$ 49.55	171.70 $\pm$ 0.43	1,336.40 $\pm$ 20.16	7.62 $\pm$ 0.25	50.86 $\pm$ 46.41	45.35 $\pm$ 0.19
Square Grid 100 $\times$ 100	C	10,000	77,474	1,326.01 $\pm$ 51.54	158.05 $\pm$ 0.78	1,323.14 $\pm$ 1.56	8.01 $\pm$ 0.21	48.74 $\pm$ 47.48	33.16 $\pm$ 0.36
Square Grid 250 $\times$ 250	BL	62,500	493,574	8,748.89 $\pm$ 108.38	899.55 $\pm$ 2.38	57,950.78 $\pm$ 848.54	53.23 $\pm$ 0.39	469.84 $\pm$ 114.43	599.20 $\pm$ 5.76
Square Grid 250 $\times$ 250	TR	62,500	493,574	8,560.52 $\pm$ 106.72	1,072.57 $\pm$ 72.21	56,140.88 $\pm$ 578.25	64.88 $\pm$ 0.66	488.90 $\pm$ 118.37	757.55 $\pm$ 5.90
Square Grid 250 $\times$ 250	C	62,500	493,574	8,724.67 $\pm$ 89.65	1,007.96 $\pm$ 69.00	56,714.30 $\pm$ 575.72	58.88 $\pm$ 0.55	426.37 $\pm$ 106.84	553.02 $\pm$ 57.02
Square Grid 500 $\times$ 500	BL	250,000	1,987,074	37,522.83 $\pm$ 270.24	3,746.24 $\pm$ 133.60	N/A	248.66 $\pm$ 0.55	3,021.86 $\pm$ 269.52	4,951.56 $\pm$ 86.97
Square Grid 500 $\times$ 500	TR	250,000	1,987,074	37,000.05 $\pm$ 226.08	4,393.82 $\pm$ 125.46	N/A	267.69 $\pm$ 0.36	3,124.22 $\pm$ 273.12	6,972.39 $\pm$ 31.20
Square Grid 500 $\times$ 500	C	250,000	1,987,074	37,301.62 $\pm$ 269.36	4,100.77 $\pm$ 170.12	N/A	275.67 $\pm$ 1.17	2,720.93 $\pm$ 252.34	4,613.97 $\pm$ 100.16
Square Grid 750 $\times$ 750	BL	562,500	4,480,574	94,151.30 $\pm$ 580.32	8,147.52 $\pm$ 237.54	N/A	596.75 $\pm$ 1.79	11,495.89 $\pm$ 853.16	16,591.28 $\pm$ 181.74
Square Grid 750 $\times$ 750	TR	562,500	4,480,574	93,459.35 $\pm$ 598.27	9,648.46 $\pm$ 210.56	N/A	661.56 $\pm$ 0.85	11,232.98 $\pm$ 780.63	26,196.78 $\pm$ 130.70
Square Grid 750 $\times$ 750	C	562,500	4,480,574	93,656.96 $\pm$ 387.81	9,000.94 $\pm$ 309.02	N/A	623.15 $\pm$ 2.27	9,689.92 $\pm$ 693.53	15,497.07 $\pm$ 164.85
Square Grid 1000 $\times$ 1000	BL	1,000,000	7,974,074	169,904.64 $\pm$ 818.12	14,925.54 $\pm$ 470.21	N/A	1,041.37 $\pm$ 3.10	31,843.25 $\pm$ 3,360.87	42,081.52 $\pm$ 803.65
Square Grid 1000 $\times$ 1000	TR	1,000,000	7,974,074	165,895.68 $\pm$ 937.99	17,447.94 $\pm$ 385.70	N/A	1,278.06 $\pm$ 3.74	30,890.19 $\pm$ 2,440.42	74,034.14 $\pm$ 240.44
Square Grid 1000 $\times$ 1000	C	1,000,000	7,974,074	169,516.68 $\pm$ 1,220.73	16,315.89 $\pm$ 417.96	N/A	1,128.19 $\pm$ 8.51	25,410.46 $\pm$ 2,179.69	38,717.81 $\pm$ 253.56

Table 2 Runtime Comparison on Rectangular Gridgraphs (Mean $\pm$ Standard Deviation ms)

Graph	Case	Nodes	Edges	BMSSP-CD	BMSSP	Vanilla Dijkstra	SCGraph MD	Nx Dijkstra	iG Dijkstra
Rectangular Grid 15 $\times$ 50	BL	750	4,984	77.58 $\pm$ 0.30	12.67 $\pm$ 0.13	7.30 $\pm$ 0.04	0.62 $\pm$ 0.07	1.61 $\pm$ 0.16	0.58 $\pm$ 0.07
Rectangular Grid 15 $\times$ 50	TR	750	4,984	68.18 $\pm$ 0.17	22.43 $\pm$ 0.14	7.38 $\pm$ 0.12	0.53 $\pm$ 0.01	1.61 $\pm$ 0.12	0.89 $\pm$ 0.11
Rectangular Grid 15 $\times$ 50	C	750	4,984	70.06 $\pm$ 0.20	16.77 $\pm$ 0.21	7.41 $\pm$ 0.12	0.53 $\pm$ 0.01	1.61 $\pm$ 0.11	0.67 $\pm$ 0.08
Rectangular Grid 25 $\times$ 50	BL	1,250	8,924	149.78 $\pm$ 0.52	19.63 $\pm$ 0.22	19.88 $\pm$ 0.03	1.06 $\pm$ 0.01	2.91 $\pm$ 0.15	1.23 $\pm$ 0.11
Rectangular Grid 25 $\times$ 50	TR	1,250	8,924	133.69 $\pm$ 0.69	30.51 $\pm$ 0.25	20.01 $\pm$ 0.05	0.94 $\pm$ 0.01	2.92 $\pm$ 0.18	1.78 $\pm$ 0.19
Rectangular Grid 25 $\times$ 50	C	1,250	8,924	143.30 $\pm$ 0.54	25.18 $\pm$ 0.43	19.97 $\pm$ 0.03	0.95 $\pm$ 0.01	2.88 $\pm$ 0.17	1.35 $\pm$ 0.14
Rectangular Grid 25 $\times$ 100	BL	2,500	17,924	299.23 $\pm$ 2.15	42.76 $\pm$ 0.40	76.76 $\pm$ 0.08	1.98 $\pm$ 0.01	5.97 $\pm$ 0.32	3.91 $\pm$ 0.37
Rectangular Grid 25 $\times$ 100	TR	2,500	17,924	265.25 $\pm$ 0.61	83.54 $\pm$ 0.62	76.78 $\pm$ 0.20	1.82 $\pm$ 0.01	6.26 $\pm$ 0.41	7.05 $\pm$ 0.73
Rectangular Grid 25 $\times$ 100	C	2,500	17,924	283.29 $\pm$ 0.81	71.31 $\pm$ 0.75	77.38 $\pm$ 0.11	1.91 $\pm$ 0.01	6.12 $\pm$ 0.38	4.67 $\pm$ 0.46
Rectangular Grid 50 $\times$ 100	BL	5,000	37,774	627.36 $\pm$ 3.62	73.07 $\pm$ 0.49	315.63 $\pm$ 0.85	3.84 $\pm$ 0.03	13.29 $\pm$ 0.63	11.16 $\pm$ 0.74
Rectangular Grid 50 $\times$ 100	TR	5,000	37,774	573.21 $\pm$ 1.80	119.27 $\pm$ 0.55	311.49 $\pm$ 0.74	3.68 $\pm$ 0.03	14.09 $\pm$ 0.82	21.61 $\pm$ 0.49
Rectangular Grid 50 $\times$ 100	C	5,000	37,774	605.55 $\pm$ 4.56	79.51 $\pm$ 0.67	316.54 $\pm$ 6.14	3.88 $\pm$ 0.02	115.78 $\pm$ 306.40	12.85 $\pm$ 0.79
Rectangular Grid 50 $\times$ 200	BL	10,000	75,774	1,250.85 $\pm$ 53.15	141.58 $\pm$ 0.57	1,307.76 $\pm$ 7.18	7.35 $\pm$ 0.26	49.26 $\pm$ 45.10	49.31 $\pm$ 0.28
Rectangular Grid 50 $\times$ 200	TR	10,000	75,774	1,124.59 $\pm$ 6.49	241.59 $\pm$ 0.89	1,274.99 $\pm$ 8.23	7.12 $\pm$ 0.31	58.04 $\pm$ 46.95	74.54 $\pm$ 0.62
Rectangular Grid 50 $\times$ 200	C	10,000	75,774	1,195.48 $\pm$ 47.75	150.05 $\pm$ 0.64	1,297.14 $\pm$ 13.33	7.50 $\pm$ 0.13	52.33 $\pm$ 46.35	54.52 $\pm$ 0.41
Rectangular Grid 100 $\times$ 500	BL	50,000	389,474	6,621.38 $\pm$ 100.14	707.64 $\pm$ 1.75	36,145.90 $\pm$ 508.87	43.66 $\pm$ 0.30	392.65 $\pm$ 104.07	476.38 $\pm$ 8.54
Rectangular Grid 100 $\times$ 500	TR	50,000	389,474	6,013.42 $\pm$ 126.09	1,251.35 $\pm$ 62.95	33,500.73 $\pm$ 357.27	46.67 $\pm$ 0.59	465.54 $\pm$ 100.17	962.73 $\pm$ 55.57
Rectangular Grid 100 $\times$ 500	C	50,000	389,474	6,149.57 $\pm$ 148.63	735.58 $\pm$ 0.94	34,934.39 $\pm$ 305.14	50.08 $\pm$ 0.42	414.50 $\pm$ 110.73	560.41 $\pm$ 20.98
Rectangular Grid 100 $\times$ 1000	BL	100,000	779,474	13,258.77 $\pm$ 182.86	1,356.78 $\pm$ 4.52	N/A	107.11 $\pm$ 0.35	1,197.74 $\pm$ 130.81	2,364.13 $\pm$ 82.27
Rectangular Grid 100 $\times$ 1000	TR	100,000	779,474	12,061.66 $\pm$ 167.19	2,456.90 $\pm$ 81.72	N/A	105.45 $\pm$ 0.30	1,493.55 $\pm$ 81.62	4,397.75 $\pm$ 61.05
Rectangular Grid 100 $\times$ 1000	C	100,000	779,474	12,359.44 $\pm$ 109.69	1,402.35 $\pm$ 4.01	N/A	94.83 $\pm$ 0.38	1,161.87 $\pm$ 112.41	2,745.41 $\pm$ 11.09
Rectangular Grid 100 $\times$ 1500	BL	150,000	1,169,474	20,002.83 $\pm$ 141.77	2,050.10 $\pm$ 93.87	N/A	150.44 $\pm$ 0.42	2,498.22 $\pm$ 201.72	5,123.66 $\pm$ 87.66
Rectangular Grid 100 $\times$ 1500	TR	150,000	1,169,474	18,147.45 $\pm$ 151.07	3,709.88 $\pm$ 125.01	N/A	171.14 $\pm$ 0.32	4,269.01 $\pm$ 338.45	10,195.76 $\pm$ 170.68
Rectangular Grid 100 $\times$ 1500	C	150,000	1,169,474	18,580.57 $\pm$ 222.69	2,089.71 $\pm$ 4.88	N/A	154.30 $\pm$ 0.55	2,612.86 $\pm$ 156.94	6,164.88 $\pm$ 52.26
Rectangular Grid 100 $\times$ 2000	BL	200,000	1,559,474	26,850.45 $\pm$ 248.77	2,691.37 $\pm$ 3.68	N/A	197.32 $\pm$ 0.47	5,016.44 $\pm$ 495.68	8,881.13 $\pm$ 177.23
Rectangular Grid 100 $\times$ 2000	TR	200,000	1,559,474	24,333.11 $\pm$ 233.06	4,924.44 $\pm$ 132.85	N/A	223.76 $\pm$ 0.40	7,647.16 $\pm$ 397.67	18,529.00 $\pm$ 154.66
Rectangular Grid 100 $\times$ 2000	C	200,000	1,559,474	24,775.74 $\pm$ 212.44	2,798.23 $\pm$ 111.36	N/A	224.18 $\pm$ 0.79	4,683.30 $\pm$ 426.53	10,864.61 $\pm$ 57.06

5.2. Break-Even Analysis: BMSSP vs. SCGraph Dijkstra

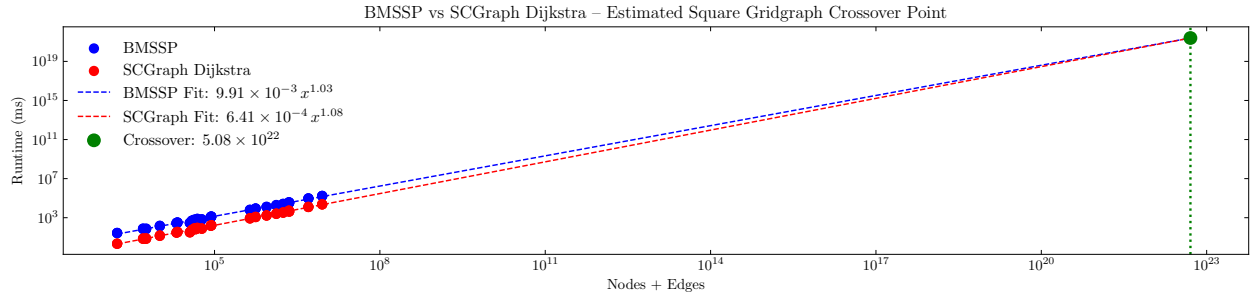
To quantify when BMSSP would overtake SCGraph Dijkstra, we model both algorithms using empirical power-law fits of the form $T(x) \approx ax^b$, where $x = n + m$ is the total graph size. The fitted curves and their intersection are shown in Figure 6. We chose power-law over claimed algorithmic time comparisons since we are not testing on worst case graphs.

Table 3 Runtime Comparison on Geospatial Graphs (Mean $\pm$ Standard Deviation ms)

Graph	Case	Nodes	Edges	BMSSP-CD	BMSSP	Vanilla Dijkstra	SCGraph MD	Nx Dijkstra	iG Dijkstra
Marnet	LA	11,062	34,698	582.78 $\pm$ 69.54	212.67 $\pm$ 0.98	1,663.45 $\pm$ 24.23	8.49 $\pm$ 1.29	45.05 $\pm$ 45.45	55.47 $\pm$ 2.26
Marnet	NY	11,062	34,698	583.47 $\pm$ 56.21	219.13 $\pm$ 3.83	1,667.02 $\pm$ 33.47	7.19 $\pm$ 0.10	44.26 $\pm$ 45.28	59.24 $\pm$ 0.64
Marnet	SEA	11,062	34,698	545.03 $\pm$ 48.86	207.56 $\pm$ 0.76	1,636.02 $\pm$ 7.76	7.24 $\pm$ 0.06	49.21 $\pm$ 53.80	69.31 $\pm$ 0.87
North America Rail	LA	9,929	28,915	505.00 $\pm$ 46.60	218.36 $\pm$ 0.98	1,289.22 $\pm$ 4.16	5.89 $\pm$ 0.09	39.93 $\pm$ 45.24	52.51 $\pm$ 0.63
North America Rail	NY	9,929	28,915	471.15 $\pm$ 1.78	230.26 $\pm$ 0.76	1,276.90 $\pm$ 2.52	5.76 $\pm$ 0.03	41.65 $\pm$ 48.25	56.16 $\pm$ 0.22
North America Rail	SEA	9,929	28,915	476.00 $\pm$ 1.82	241.48 $\pm$ 45.82	1,279.13 $\pm$ 1.31	5.77 $\pm$ 0.05	39.50 $\pm$ 43.45	48.64 $\pm$ 0.30
Oak Ridge Maritime	LA	10,661	25,036	289.94 $\pm$ 1.74	191.42 $\pm$ 0.38	1,464.56 $\pm$ 15.11	5.78 $\pm$ 0.05	36.61 $\pm$ 51.34	31.66 $\pm$ 0.81
Oak Ridge Maritime	NY	10,661	25,036	299.22 $\pm$ 45.94	186.95 $\pm$ 0.60	1,478.70 $\pm$ 12.10	5.57 $\pm$ 0.03	35.29 $\pm$ 42.94	39.53 $\pm$ 0.32
Oak Ridge Maritime	SEA	10,661	25,036	290.30 $\pm$ 0.72	192.69 $\pm$ 0.46	1,457.89 $\pm$ 7.87	5.67 $\pm$ 0.11	39.72 $\pm$ 48.55	52.68 $\pm$ 0.14
US Freeway	LA	14,591	44,232	657.61 $\pm$ 3.54	349.40 $\pm$ 45.25	2,819.43 $\pm$ 30.46	7.13 $\pm$ 0.10	78.79 $\pm$ 46.35	186.30 $\pm$ 2.73
US Freeway	NY	14,591	44,232	663.95 $\pm$ 1.91	352.63 $\pm$ 45.68	2,823.42 $\pm$ 41.62	7.19 $\pm$ 0.20	69.31 $\pm$ 43.45	148.48 $\pm$ 2.04
US Freeway	SEA	14,591	44,232	676.26 $\pm$ 2.78	348.28 $\pm$ 46.70	2,814.11 $\pm$ 18.30	7.37 $\pm$ 0.38	88.15 $\pm$ 44.69	218.86 $\pm$ 5.71
World Highways and Marnet	LA	572,009	1,689,541	33,465.90 $\pm$ 401.34	14,639.99 $\pm$ 178.77	N/A	476.68 $\pm$ 1.97	8,261.05 $\pm$ 462.97	17,028.90 $\pm$ 568.46
World Highways and Marnet	NY	572,009	1,689,541	33,202.17 $\pm$ 304.31	15,287.75 $\pm$ 212.13	N/A	468.30 $\pm$ 0.88	6,470.21 $\pm$ 452.73	16,459.89 $\pm$ 177.08
World Highways and Marnet	SEA	572,009	1,689,541	33,502.62 $\pm$ 441.96	14,514.57 $\pm$ 164.47	N/A	481.25 $\pm$ 1.70	5,530.60 $\pm$ 350.42	12,222.92 $\pm$ 172.31

BMSSP is well-approximated by $T_{\text{BMSSP}}(x) \approx 9.91 \times 10^{-3} x^{1.03}$, while SCGraph Dijkstra follows $T_{\text{SCGraph}}(x) \approx 6.41 \times 10^{-4} x^{1.08}$. BMSSP grows slightly more slowly with x , but SCGraph has a much smaller constant factor. Solving for the intersection of these two fitted curves yields a crossover point of approximately $x^* \approx 5.08 \times 10^{22}$.

This crossover is far beyond any reasonably loadable graph size for a computer in the foreseeable future. As a reference point, the full OpenStreetMap planet graph contains on the order of 10^8 nodes and edges combined, which is many orders of magnitude smaller than x^* . Although BMSSP has the marginally better asymptotic exponent, SCGraph Dijkstra remains faster for all practically realizable sparse graphs. In practice, system-level optimizations dominate, and BMSSP’s theoretical advantage emerges only at scales that cannot be reached in today’s memory or storage.

Figure 6 BMSSP vs SCGraph Dijkstra – Estimated Square Gridgraph Crossover Point

6. Conclusion

The Bounded Multi-Source Shortest Path (BMSSP) algorithm of Duan et al. (2025) is a major theoretical breakthrough: it breaks the Dijkstra sorting barrier and achieves a deterministic $O(m \log^{2/3} n)$ time for SSSP on graphs with nonnegative weights. However,

until now, this advance has remained largely theoretical, with no public implementation and several key steps left ambiguous in practical settings.

This paper takes a first step toward closing that gap by introducing **BMSSPy**, the first functional, open-source Python implementation of **BMSSP**. Our robust, reproducible implementation does more than translate pseudocode into code: it addresses potential bugs, relaxes unrealistic assumptions, and makes implicit components, such as tight-edge forest construction, pivot selection, the underlying data structures, and the set operations, fully explicit. We add features such as stable predecessor tracking, handling unreachable nodes, and irregular degree distributions, turning **BMSSP** into an end-to-end algorithm that returns both distances and routes on real networks.

We complement this with comprehensive empirical evaluation on synthetic gridgraphs and real geospatial networks, comparing **BMSSP** to Dijkstra implementations in **NetworkX**, **igraph**, and the highly optimized **SCGraph**. **BMSSP** clearly outperforms a vanilla Python Dijkstra and becomes competitive with general-purpose libraries on large or elongated grids, but **SCGraph**’s Dijkstra remains substantially faster across all realistic instances. Our break-even analysis shows that the asymptotic advantage of **BMSSP** would materialize only at astronomically large graph sizes, consistent with the overhead concerns already noted by Duan et al. (2025).

While our current implementation prioritizes algorithmic clarity and theoretical fidelity, this choice naturally incurs runtime and memory overheads compared to optimized libraries like **SCGraph** and **igraph**. Consequently, **BMSSPy** currently serves best as a research instrument and a reference “blueprint” rather than a drop-in replacement for shortest path implementations. However, we view this as a necessary and exciting milestone: by successfully verifying the complex logic and data structures in Python, we lay a solid foundation for future high-performance ports in other languages. We are optimistic that future research and implementations will dramatically reduce memory footprints and constant-factor delays, allowing the algorithm’s asymptotic power to shine in practice.

Narrowing this gap will require reducing constant factors. This may be achieved through changing the constant degree requirements of the graph, work budget modifications, faster data structures, refined pivot and window policies, or hybrid strategies that combine **BMSSP**’s bounded-range logic with other heuristics. We hope that **BMSSPy** will serve as a platform for this next step. The package is fully documented, reproducible, and designed

to support future research and implementations. By providing the first complete, open, and practical implementation of BMSSP, BMSSPy offers a foundation for future algorithmic improvements and makes the BMSSP framework accessible to researchers and practitioners in operations research, transportation, and network analytics.

References

- Bellman R (1958) On a routing problem. *Quarterly of Applied Mathematics* 16(1):87–90.
- Castro L, Clementino T, de Freitas R (2025) Implementation and brief experimental analysis of the Duan et al.(2025) algorithm for single-source shortest paths. *arXiv preprint arXiv:2511.03007* .
- Chen M, Chowdhury RA, Ramachandran V, Roche DL, Tong L (2007) Priority queues and dijkstra’s algorithm. *UTCS Technical Report* TR-07-54.
- Csardi G, Nepusz T (2006) The igraph software package for complex network research. *InterJournal, Complex Systems* 1695.
- Dijkstra EW (1959) A note on two problems in connexion with graphs. *Numerische Mathematik* 1:269–271.
- Dreyfus SE (1969) An appraisal of some shortest-path algorithms. *Operations Research* 17(3):395–412.
- Driscoll JR, Gabow HN, Shrairman R, Tarjan RE (1988) Relaxed heaps: An alternative to fibonacci heaps with applications to parallel computation. *Communications of the ACM* 31(11):1343–1354.
- Duan R, Mao J, Mao X, Shu X, Yin L (2025) Breaking the sorting barrier for directed single-source shortest paths. *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, 36–44.
- Elmaghraby SE (1970) The theory of networks and management science. part i. *Management Science* 17(1):1–34.
- Ford J Lester R (1956) Network flow theory. Technical Report P-923, RAND Corporation, Santa Monica, California.
- Fredman ML, Tarjan RE (1987) Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM (JACM)* 34(3):596–615.
- Golden B (1976) Shortest-path algorithms: A comparison. *Operations Research* 24(6):1164–1168.
- Haeupler B, Hladík R, Rozhoň V, Tarjan RE, Tetěk J (2024) Universal optimality of dijkstra via beyond-worst-case heaps. *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, 2099–2130 (IEEE).
- Hagberg A, Swart PJ, Schult DA (2008) Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States).
- Hart PE, Nilsson NJ, Raphael B (1968) A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4(2):100–107.

-
- Makowski C (2025) Bmsspy: A pure python bmssp solver for python. <https://github.com/connor-makowski/bmsspy>, python package, accessed 2025-09-21.
- Makowski C, Saragih A, Guter W, Russell T, Heinold A, Lekkakos S (2025) Scgraph: A dependency-free python package for road, rail, and maritime shortest path routing generation and distance estimation.
- Orlin JB, Madduri K, Subramani K, Williamson M (2010) A faster algorithm for the single source shortest path problem with few distinct positive lengths. *Journal of Discrete Algorithms* 8(2):189–198.
- Thorup M (1999) Undirected single-source shortest paths with positive integer weights in linear time. *Journal of the ACM (JACM)* 46(3):362–394.
- Thorup M (2003) Integer priority queues with decrease key in constant time and the single source shortest paths problem. *Proceedings of the Thirty-fifth Annual ACM Symposium on Theory of Computing*, 149–158.
- Valko D, Paranjpe R, Gómez JM (2025) Outperforming dijkstra on sparse graphs: The lightning network use case. *arXiv preprint arXiv:2509.13448* .
- Zhan FB, Noon CE (1998) Shortest path algorithms: an evaluation using real road networks. *Transportation science* 32(1):65–73.

Appendix

Table 4: Notation used throughout the paper.

Notation	Description
Graph Structure	
$G = (V, E)$	Directed graph with vertex set V and edge set E .
V	Set of vertices.
E	Set of edges.
$n = V $	Number of vertices.
$m = E $	Number of edges.
$u, v \in V$	Individual vertices.
w	Nonnegative weight of edge (u, v) .
Distance, Paths, and Predecessors	
$\text{dist}[v]$	Tentative shortest-path distance from the source to vertex v .
$\text{cdist}[v]$	Counter-adjusted effective distance for tie-breaking.
$\text{pred}[v]$	Predecessor of vertex v in the shortest-path tree.
path	Ordered list of vertices forming the shortest path.
length	Total distance of the shortest path.
∞	Infinity; indicates unreachable vertices.
-1	Sentinel indicating no predecessor or unreachable nodes.
Origins and Destinations	
O	Set of origin vertices.
o	Origin vertex.
dest	Destination vertex.
Frontier and Recursion Sets	
S	Frontier set at the current recursion level.
S'	New frontier set.
S_{curr}	The current frontier subset processed.
U	Set of vertices that exceed the distance bound or remain unprocessed.

Notation	Description
U_{curr}	Newly discovered vertices added to U during the current iteration of Algorithm 3.
W	Expanded frontier set containing vertices with $\text{dist} < B$.
W_{prev}	Previous frontier set in relaxation iterations.
W_{curr}	Current frontier set during relaxations.
P	Set of pivot vertices chosen for the next recursion level.
F	Directed forest over vertices in W , containing edges $(u \rightarrow v)$ where $\text{pred}[v] = u$. Each vertex has at most one parent, so F is a directed acyclic graph (DAG). Used for subtree computations and pivot selection.
F^{ID}	Vertices in this set have an indegree in the frontier; roots are not stored in this set.
Distance Bounds	
B	Current global distance bound.
B' or B_{new}	Updated distance bound after refinement.
B_{curr}	Active window's distance bound at a recursion stage.
B_{comp}	Completion bound for fully processed vertices.
Algorithm Parameters	
$k = \lfloor (\log_2 n)^{1/3} \rfloor$	Controls frontier-size threshold and pivot sampling.
$t = \lfloor (\log_2 n)^{2/3} \rfloor$	Controls subtree-size threshold.
$l = \lceil \log_2(n)/t \rceil$	Recursion depth; number of hierarchical levels.
$M = 2^{l-1}t$	Capacity parameter; work budget per recursion level.
<i>precision</i>	Precision parameter for edge weight rounding.
Tie-Breaking and Structural Metadata	
<i>counter</i>	Global tie-break counter ensuring unique effective distances.
$G_{\text{edge}}[u][v]$	Unique per-edge adjustment used to break ties in distance computations. Ensures lexicographic ordering of effective distances in BMSSPy's relaxations.
Heap	Binary min-heap used for priority queue operations.

Algorithm 6: Is Pivot. This helper function determines whether a given node qualifies as a pivot by checking the size of its reachable subtree within a directed forest F . Using a simple depth-first traversal, it counts visited nodes until reaching the threshold t , returning **True** if the limit is met and **False** otherwise. This ensures efficient identification of influential frontier roots during pivot selection.

Algorithm 6: Is Pivot (is_pivot)

Input: root r , directed forest F (adjacency list: $F[v] = \text{children of } v$), threshold t

Output: **True** if number of nodes reachable from r is $\geq t$; **False** otherwise

```

1  $seen \leftarrow \emptyset$ ;  $stack \leftarrow [r]$ ;  $cnt \leftarrow 0$ ;
2 while  $stack \neq \emptyset$  do
3    $x \leftarrow \text{POP}(stack)$ ;
4   if  $x \in seen$  then
5     continue
6    $cnt \leftarrow cnt + 1$ ;
7   if  $cnt \geq t$  then
8     return True
9    $seen \leftarrow seen \cup \{x\}$ ;
10  foreach  $y \in F[x]$  do
11     $\text{PUSH}(stack, y)$ ;
12 return False;

```

Algorithm 7: Reconstruct Path. This utility function rebuilds the full shortest path from the destination node d back to its origin using the predecessor array P . It iteratively traces backward until the source is reached, reverses the sequence, and returns the ordered path. This produces an explicit route corresponding to the computed shortest-path distance.

Algorithm 7: Reconstruct Path (reconstruct_path)

Input: destination node $dest$, predecessor list $pred$

Output: list L representing the reconstructed path from origin to $dest$

```

1  $L \leftarrow [dest]$ ;
2 while  $pred[dest] \neq -1$  do
3    $dest \leftarrow pred[dest]$ ;
4    $\text{append } dest \text{ to } L$ ;
5  $\text{REVERSE}(L)$ ;
6 return  $L$ ;

```

Algorithm 8: Convert to Constant Out-Degree. This utility function transforms an arbitrary directed graph into an equivalent representation in which every node has at most a specified out-degree d . Whenever a node exceeds this limit, it is expanded into a sequence of auxiliary nodes, each responsible for at most $d - 1$ of the original outgoing edges. These auxiliary nodes are then linked in a zero-weight cycle to preserve reachability and allow traversal through all outgoing edges of the original node. The function returns the augmented graph, a mapping that records the correspondence between new and original nodes, and the size of the original graph. This conversion enables algorithms that require bounded out-degree without altering the graph's essential connectivity or path-weight structure.

Algorithm 8: Convert to Constant Out-Degree (`convert_to_constant_out_degree`)

Input: graph G (adjacency maps), target out-degree d
Output: dictionary with converted graph, index map, and original graph length

```

1  $n \leftarrow |G|$ ;  $G' \leftarrow G$ ;  $M \leftarrow [0, 1, \dots, n - 1]$ ;
2 for  $u \leftarrow 0$  to  $n - 1$  do
3   if  $|G[u]| > d$  then
4      $k \leftarrow \left\lceil \frac{|G[u]|}{d-1} \right\rceil$ ;                                // number of partitions
5      $parts \leftarrow [u, |G'|, |G'| + 1, \dots, |G'| + k - 2]$ ;
6     extend  $G'$  with  $(k - 1)$  empty maps;
7     extend  $M$  with  $(k - 1)$  copies of  $u$ ;
8      $G'[u] \leftarrow \{\}$ ;                                // clear existing outgoing edges
9      $i \leftarrow 0$ ;  $r \leftarrow 0$ ;
10    foreach  $(v, w) \in G[u]$  do
11       $G'[parts[i]][v] \leftarrow w$ ;
12       $r \leftarrow r + 1$ ;
13      if  $r \geq d - 1$  then
14         $r \leftarrow 0$ ;  $i \leftarrow i + 1$ ;
15    // Add cycle edges of weight 0 between all partition nodes
16    for  $j \leftarrow 0$  to  $k - 1$  do
17       $G'[parts[j]][parts[(j + 1) \bmod k]] \leftarrow 0$ ;
18 return  $C \leftarrow \{graph \leftarrow G', idx\_map \leftarrow M, original\_graph\_len \leftarrow n\}$ ;

```

Algorithm 9: Convert From Constant Out-Degree. This utility function reverses the transformation applied during constant out-degree conversion, restoring shortest-path data from the augmented graph back to the original graph. Using the index map produced in the forward conversion, each predecessor entry in the constant-degree graph is traced backward until a valid predecessor from the original graph is identified. Auxiliary nodes introduced solely to enforce bounded out-degree are skipped transparently, ensuring that no artificial structure influences the recovered paths. Distances are truncated to the original graph

size, and a new predecessor matrix is constructed in which every predecessor refers only to an authentic node of the initial graph. The function returns the reconstructed distance and predecessor matrices, thereby reestablishing shortest-path information in the original graph topology.

Algorithm 9: Convert from Constant Out-Degree (`convert_from_constant_out_degree`)

Input: dist_{cd} (distance matrix), pred_{cd} (predecessor matrix), constant-degree dictionary C

Output: dictionary with converted dist and pred matrices

```

1  $M \leftarrow C[\text{idx\_map}]$ ;
2  $n \leftarrow C[\text{original\_graph\_len}]$ ;
3  $\text{pred} \leftarrow []$ ; // initialize empty predecessor matrix
4 for  $i \leftarrow 0$  to  $n - 1$  do
5    $u \leftarrow \text{pred}_{\text{cd}}[i]$ ;
6   while true do
7     if  $u = -1$  then
8       append  $u$  to  $\text{pred}$ ;
9       break;
10    else
11      if  $M[u] < n$  and  $M[u] \neq i$  then
12        append  $M[u]$  to  $\text{pred}$ ;
13        break;
14      else
15         $u \leftarrow \text{pred}_{\text{cd}}[u]$ ; // follow predecessor chain
16 return  $\{ \text{distance\_matrix} \leftarrow \text{dist}_{\text{cd}}[0:n], \text{predecessor\_matrix} \leftarrow \text{pred} \}$ ;

```
